CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

(CAST: Carbon-Aware Spatio-Temporal Scheduling under Capacity Constraints)

요 약 탄소 인지 스케줄링은 작업을 탄소집약도가 낮은 시각과 리전으로 옮겨 배출을 줄인다. 그러나 모든 작업이 같은 저탄소 시각과 리전을 고르므로, 리전 용량을 고려하지 않으면 한 리전에 몰린다. 본 논문은 마감과 리전 용량을 지키며 배출을 줄이는 스케줄러 CAST를 제안한다. CAST는 매시간 모든 대기 작업의 리전과 시작 시각을 하나의 선형 계획으로 다시 풀어 그 시간에 시작할 작업만 확정하고, 탄소와 왕복 지연 시간 사이의 가중치는 무릎점 규칙으로 고른다. 8개 리전의 2025년 평균 배출계수로 계산하면, CAST는 합성 워크로드의 배출을 71.93%, Azure VM 기록의 배출을 51.35% 줄였고 상한과 마감을 모두 지켰다. 탄소만 보면 CAST는 95.65%를 줄여, 1년치 탄소집약도를 미리 알지만 작업을 옮기지 않을 때의 9.9배 용량이 필요한 Sukprasert 등의 정책보다 0.98%p 덜 줄였다. Caspian은 CAST보다 덜 줄이면서 5,582건의 마감을 어겼다.

키워드 : 탄소 인지 스케줄링, 시공간 워크로드 이동, 리전 용량 제약, 클라우드 컴퓨팅, 선형 계획

Abstract Carbon-aware scheduling reduces emissions by moving batch jobs to hours and regions with low carbon intensity. However, every job picks the same low-carbon hours and regions, so jobs pile up in one region when regional capacity is ignored. This paper presents CAST, a scheduler that reduces emissions while meeting deadlines and regional capacity. Every hour, CAST re-solves a single linear program over the regions and start times of all waiting jobs, commits only the jobs that start in that hour, and selects the weight between carbon and round-trip latency with a knee-point rule instead of a preset value. With average emission factors of eight regions in 2025, CAST reduced emissions by 71.93% on a synthetic workload and by 51.35% on a workload built from Azure VM traces, without exceeding capacity or missing deadlines. With carbon as the only objective, CAST came within 0.98 percentage points of the Sukprasert et al. policy, which knows the whole year of carbon intensity in advance but needs 9.9 times the capacity of running every job in place. Caspian reduced emissions less than CAST and missed 5,582 deadlines.

Key words : Carbon-aware scheduling, Spatio-temporal workload shifting, Regional capacity constraint, Cloud computing, Linear programming

1. 서론

전 세계 데이터센터의 전력 소비는 2024 년 약 415 TWh 에서 2030 년 945 TWh 로 늘 것으로 전망되고[1], 유럽연합은 일정 규모 이상의 데이터센터에 에너지 사용 보고를 의무화했다[2]. 기존 설비에 바로 적용할 수 있는 방법 중 하나가 작업을 탄소 배출이 적은 시각과 리전(클라우드 사업자가 데이터센터를 묶어 운영하는 지역 단위)으로 옮기는 탄소 인지 스케줄링이다. 전력 1 kWh 를 생산할 때 배출되는 CO_2 양인 탄소집약도는 발전원 구성에 따라 시각과 지역마다 다르다. 본 연구의 8 개 리전에서 2025 년 평균은 인도 543.2, 프랑스 13.6 gCO_2/kWh 로 약 40 배 차이 난다. 마감 안에만 끝나면 되는 배치 작업은 이런 저탄소 시각과 리전으로 옮길 수 있다.

선행 연구는 이 차이를 시간 축(CICS[3])이나 공간 축(CASPER[4], VMware 의 전역 부하 분산[5])에서 활용해 왔다. 두 축을 함께 다룬 연구에는 리전 용량이 문제가 된다. 탄소집약도가 가장 낮은 리전과 시각은 모든 작업에 같으므로, 작업이 한곳에 몰려 리전이 동시에 실행할 수 있는 작업 수를 넘기기 쉽다. Sukprasert 등[6]은 두 축을 결합해 절감의 최대치를 구한 분석에서 리전 용량을 두지 않았다. 이 정책을 본 연구의 합성 워크로드에 그대로 적용하면 작업 건수의 99.75%가 프랑스 한 리전에 몰린다. 이 배정을 위반 없이 실행하려면 한 리전이 작업 129 개를 동시에 실행해야 하며, 이는 모든 작업을 제출 리전에서 바로 실행할 때의 최대 동시 실행 수(기준 피크 13)의 9.9 배다. 용량을 다룬 Caspian[7]은 마감을 목적함수의 페널티로 완화하고, 용량과 마감을 함께 지키는 Zanotto 등[8]은 작업이 도착할 때 배정을 확정한 뒤 다시 조정하지 않는다.

본 논문은 마감을 지키면서 리전 용량 안에서 배출을 줄이는 스케줄러 CAST(Carbon-Aware Spatio-Temporal scheduler)를 제안한다. CAST 는 1 시간 슬롯마다 대기 중인 모든 작업의 리전과 시작 시각을 하나의 선형 계획 문제로 다시 풀고, 이번 슬롯에 시작하는 작업만 확정한다. 마감은 하드 제약으로 지키고, 용량은 상한을 넘긴 양에 가장 큰 페널티를 주는 소프트 제약으로 둔다. 탄소집약도는 리전마다 LSTM 으로 24 시간 앞까지 예측하고, 탄소와 왕복 지연 시간 사이의 가중치는 미리 정하지 않고 슬롯마다 무릎점 규칙으로 고른다.

8 개 리전의 2025 년 탄소집약도에서 평균 배출계수로 배출을 계산하면, CAST 는 합성 워크로드의 배출을 71.93%, Azure VM 기록으로 만든 워크로드의 배출을 51.35% 줄였고, 두 워크로드 모두 상한 위반과 마감 위반이 없었다. 탄소만 고려하면 CAST 는 상한 안에서 95.65%를 줄여 1 년치를 미리 아는 Sukprasert 등의 정책보다 0.98%p 덜 줄였다. Caspian 은 CAST 보다 적게 줄이면서 작업 5,582 건의 마감을 어겼다.

2 장은 관련 연구, 3 장은 CAST 의 구성, 4 장은 실험 결과, 5 장은 결론과 향후 연구를 다룬다.

2. 관련 연구

탄소 인지 스케줄링은 시간 축에서 먼저 연구되었다. Wiesner 등은 한 데이터센터 안에서 작업을 미루는 것만으로 얻는 절감을 보였고[9], CarbonScaler 는 한 리전 안에서 작업의 서버 수를 탄소 예보에 맞춰 늘리고 줄이며[10], GAIA 는 한 리전 클러스터에서 탄소, 완료 시간, 비용 사이의 상충 관계를 다룬다[11]. 이 연구들은 작업이 실행되는 리전을 바꾸지 않는다. 한편 요청과 배치 작업을 데이터센터 사이로 나눠 보내거나[12-14] 한 데이터센터 안에서 재생에너지가 많은 시각으로 미뤄[15] 전기요금이나 화석 에너지 사용을 줄이는 연구도 있었다. 리전과 시각을 함께 옮기면서 리전 용량과 마감을 지킨다는 관점에서, 선행 연구에는 다음 세 가지 한계가 있다.

첫째, 리전과 시각을 함께 옮기는 연구는 리전 용량과 마감 가운데 하나를 제약으로 두지 않는다. Sukprasert 등은 용량이 공간 이동의 이득을 좌우함을 확인하고도 두 축을 결합한 분석에서는 용량을 두지 않았고[6], Attenni 등도 자원이 항상 가용하다고 가정한다[16]. ST-CLIP 은 작업 하나를 마감 안에서 리전 사이로 옮기는 온라인 문제를 다루며, 여러 작업의 자원 공유와 데이터센터 용량은 고려하지 않는다[17]. Caspian 은 반대로 용량을 하드 제약으로 두고 마감을 목적함수의 페널티로 완화한다[7].

둘째, 용량을 고려하는 연구는 대개 작업이 도착할 때 배정을 한 번 정하고 다시 조정하지 않는다. Zanotto 등은 리전별 동시 실행 상한과 작업별 하드 마감을 함께 두고, 요청이 도착할 때마다 이미 정한 배정을 고려해 그 요청의 리전과 시각을 정수 계획으로 정하고, 이후 다시 풀지 않는다[8]. CICS 는 하루 전에 산정한 자원 곡선으로 다음 날의 유연한 부하를 묶어 개별 작업의 마감을 보지 않으며[3], Flex 는 단일 클러스터 안에서 용량을 보지 않는 시간 이동이 저탄소 시각에 부하를 모은다는 점을 지적했으나 리전 사이의 이동은 다루지 않는다[18]. 매 기간 이후 구간 전체를 다시 최적화하고 당장의 결정만 실행하는 롤링 호라이즌 방식은 생산 계획과 모형 예측 제어의 표준 구조다[19–21]. 확정 시점으로 구분하면 Zanotto 등의 방식은 도착 시 확정, CAST 는 시작 시 확정에 해당한다[22].

셋째, 탄소와 그 밖의 목적 사이의 가중치를 사람이 정하며, 값을 고르는 방법은 다루지 않는다. CASPER 는 탄소와 서버 프로비저닝 비용 사이의 가중치를 고정하고 다른 값에서는 결과가 달라진다고 밝혔다[4]. CICS 도 탄소 비용과 피크 전력 비용의 가중합을 쓰며[3], 두 축을 하나의 최적화 문제로 푸는 Caspian 도 가중치를 사람이 정한다[7]. 다목적 최적화에서는 모든 목적값이 최소인 이상점(ideal point)에 가장 가까운 해를 고르는 방법이 쓰인다[23].

표 1 은 선행 연구의 설계를 비교한 것이다. ○는 있음, △는 페널티나 부분 분석, ×는 없음, —는 해당 없음을 뜻한다.

표 1 선행 연구의 설계 비교

Table 1 Design properties of prior work

Work	Time	Space	Region cap	Hard deadline	Re-solve
Wiesner, CarbonScaler, GAIA [9–11]	○	×	×	△	×
CICS [3]	○	×	△	×	×
CASPER [4]	×	○	△	—	—
Sukprasert [6]	○	○	△	○	×
ST-CLIP [17]	○	○	×	○	—
Flex [18]	○	×	○	△	○
Caspian [7]	○	○	○	△	○
Zanotto [8]	○	○	○	○	×
CAST	○	○	△	○	○

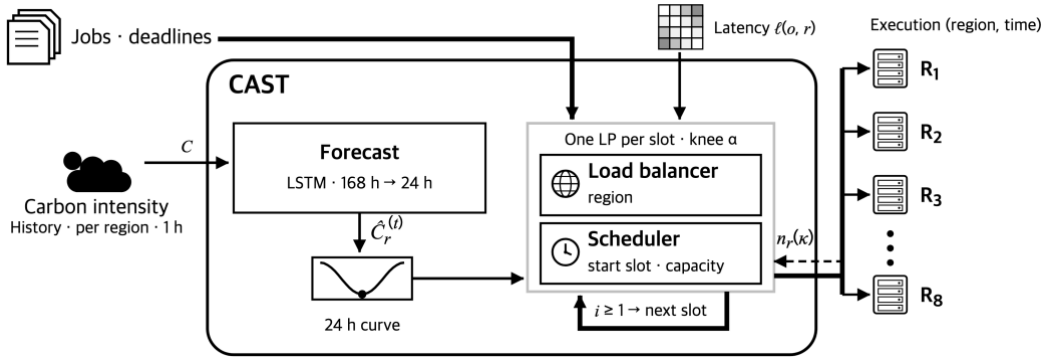


그림 1 CAST 의 구성

Fig. 1 Overview of CAST

3. CAST

CAST 는 2 장에서 본 세 가지 한계에 각각 다음과 같이 대응한다. 첫째, 마감은 반드시 지키고 용량은 상한을 넘긴 양에 가장 큰 페널티를 준다. 둘째, 1 시간 슬롯마다 대기 중인 모든 작업의 리전과 시작 시각을 하나의 선형 계획 문제로 다시 풀고 이번 슬롯에 시작하는 작업만 확정하는 롤링 호라이즌(rolling horizon) 방식[19–21]을 채택한다. 셋째, 탄소와 지연 사이의 가중치를 미리 정하지 않고, 슬롯마다 무릎점 규칙으로 고른다.

3.1 문제 정의

그림 1 은 CAST 의 구성이다. 탄소집약도 예측기는 리전마다 앞으로 24 시간의 값을 예측한다. 로드밸런서는 실행할 리전의 후보를, 스케줄러는 시작 시각의 후보를 만들고, 통합 배정은 두 후보를 묶어 하나의 선형 계획 문제로 함께 고른다. 이번 슬롯에 시작하는 작업만 확정되어 리전별 실행 현황에 반영되고, 다음 슬롯에서는 이 현황으로 리전별 남은 용량을 계산한다.

작업 j 는 제출 리전 o_j, 제출 시각 s_j, 실행 시간 d_j, 허용 대기 시간 L_j 를 가지며, 마감은 $D_j = s_j + L_j + d_j$ 다. 결정 변수는 실행 리전 $r(j)$와 시작 시각 τ_j 다. 총 배출량 E는 다음과 같다.

$$E = \sum_{j \in J} P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t)\,dt \qquad (1)$$

$C_r(t)$는 리전 r 의 실측 탄소집약도이고, P 는 작업당 소비 전력이다. 작업별 전력 자료가 없으므로 모든 작업의 P 를 1 kW 로 두고, 모든 정책에 같은 값을 적용한다. 전력이 같으면 P 는 절감률에서 약분된다. 배정은 예측값으로 정하고 배출량은 실측값으로 계산한다. 실측 자료가 1 시간 단위이므로 적분은 실행이 걸친 슬롯들의 평균 탄소집약도에 실행 시간을 곱해 계산한다. 첫 번째 제약은 마감이다.

$$s_j \le \tau_j, \quad \tau_j + d_j \le D_j \qquad \forall j \in J \qquad (2)$$

식 (2)는 작업이 제출 이후 시작해 마감 안에 끝나야 함을 뜻하는 하드 제약(hard constraint)이다. 두 번째 제약은 용량이다. 리전 r 에서 시각 t 에 실행 중인 작업 수를 $n_r(t)$, 한 사용자나 클러스터가 리전 r 에서 동시에 실행할 수 있는 작업 수의 상한을 cap_r 로 쓴다.

$$n_r(t) \le \mathrm{cap}_r + z_r(t),\ z_r(t) \ge 0 \quad \forall r \in R, \forall t \qquad (3)$$

$z_r(t)$는 상한 초과량이다. 용량은 초과량에 페널티를 주는 소프트 제약(soft constraint)으로 두며, 이 페널티를 목적함수에서 가장 크게 둔다. 따라서 용량이 부족한 순간에도 모든 작업이 마감을 지키며, 상한을 넘기는 배정은 다른 리전과 시각에 남은 용량이 없을 때 생기도록 하였다.

3.2 탄소집약도 예측기

탄소집약도 예측기는 리전마다 따로 학습한 2 층 LSTM 이다(그림 2). hidden size 는 64, dropout 은 0.2 다. 과거 168 시간의 탄소집약도, 무탄소·재생 전력 비율, 시각·요일·월, 공휴일 여부(10 개)를 입력받아 앞으로 24 시간을 한 번에 예측한다. 캘리포니아, 텍사스, 독일은 과거 풍속, 일사량, 기온을 함께 입력받고, 앞으로 24 시간의 기상값을 출력층에 더한다. 학습 설정과 예측 성능은 4.1.1 절에 있다.

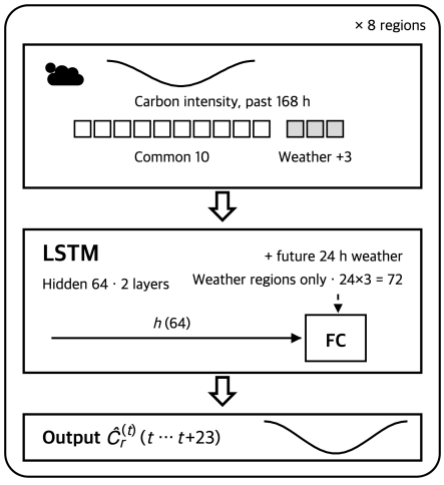


그림 2 리전별 LSTM 탄소집약도 예측기의 구조

Fig. 2 Per-region LSTM carbon intensity forecaster

3.3 로드밸런서

로드밸런서는 작업마다 실행할 수 있는 리전 후보와 그 지연 비용을 정한다. 후보는 8 개 리전이며, 리전 간 왕복 지연 시간(RTT)은 표 2 와 같다. 값은 Azure 의 리전 간 왕복 지연 통계[24]에서 가져왔고 1 년 동안 고정한다. 뉴욕은 Azure 리전이 없어 가장 가까운 버지니아의 값을 쓴다. 작업마다 왕복 지연 시간의 상한을 둘 수 있으며, 그보다 먼 리전은 후보에서 뺀다. 주 실험은 이 상한을 두지 않으며, 상한을 둔 결과는 4.4.4 절에 있다.

작업을 제출 리전이 아닌 곳에서 실행하는 동안에는 자료가 리전 사이를 오가고, 그 경로의 장애에도 노출된다. 그래서 왕복 지연 시간 $\ell(o,r)$을 비용으로 두고 실행 시간에 비례하게 매긴다(3.6 절). 지연 비용은 왕복 지연 시간을 표 2 의 최댓값 244 ms 로 나눈 값이다.

$$\tilde{\ell}(o,r) = \frac{\ell(o,r)}{244} \tag{4}$$

표 2 리전 간 왕복 지연 시간(ms)

Table 2 Inter-region round-trip latency (ms)

Region	CA	TX	NY	FR	DE	KR	JP	IN
CA(California)	0	39	74	146	153	134	109	225
TX(Texas)	39	0	38	113	125	153	128	244
NY(New York)	74	38	0	90	95	193	172	204
FR(France)	146	113	90	0	12	238	228	137
DE(Germany)	153	125	95	12	0	244	235	142
KR(Korea)	134	153	193	238	244	0	30	122
JP(Japan)	109	128	172	228	235	30	0	126
IN(India)	225	244	204	137	142	122	126	0

3.4 스케줄러

스케줄러는 작업의 시작 시각 후보를 만들고, 각 (리전, 시작 시각) 쌍의 탄소 비용을 정한다. 슬롯 t 가 시작하는 정각에 아직 시작하지 않은 작업과 이번 슬롯 안에 제출될 작업을 모아 대기 작업 집합 Q 를 만든다(4.1.1 절).

스케줄러의 후보는 작업 j 를 현재 슬롯부터 몇 슬롯 뒤에 시작할지를 나타내는 i 다. i=0 은 이번 슬롯에 시작하는 것으로, 이미 기다리던 작업은 정각에, 슬롯 중간에 제출되는 작업은 제출 순간에 시작한다. i≥1 은 슬롯 t+i 의 정각에 시작한다. i 는 마감 안에 끝낼 수 있는 마지막 시작 시각과 예측 구간 H(24 시간) 가운데 이른 쪽까지 둔다. 제출된 슬롯에서만 시작할 수 있는 작업을 연기 불가 작업이라 한다. 이 작업과 마지막 시작 시각까지 1 시간이 남지 않은 작업은 i=0 만 갖는다.

시작 후보의 탄소 비용은 그 시각부터 실행하는 동안의 예측 탄소집약도 평균을 슬롯 t 예측 전체의 최댓값으로 나눈 값이다.

$$\tilde{C}_j(r,i) = \frac{\hat{C}_r^{(t)}\left(t+i, d_j\right)}{\max_{r',t'} \hat{C}_{r'}^{(t)}(t')} \tag{5}$$

$\hat{C}_r$(t)(t+i, d_j)는 슬롯 t 에 나온 예측으로 구한, 리전 r 에서 t+i 부터 d_j 동안의 평균 탄소집약도다.

3.5 용량 관리

CAST 는 리전 용량을 작업 하나가 들어올 때 확인하지 않고, 대기 중인 모든 작업이 함께 나눠 쓰는 제약으로 둔다. 동시 실행 수는 이번 슬롯 안에서는 작업이 시작하는 시점(정각과 슬롯 중간의 제출 시각, 집합 U_t)마다, 그 이후에는 미래 슬롯 $\sigma \in [t+1, t+H]$마다 센다. 이 시점들을 κ 로 묶어 쓰면, 이미 시작한 작업 수 $n_r(\kappa)$와 새로 배정될 작업 수의 합이 상한을 넘은 만큼이 초과량 z^A 다.

$$\sum_{(j,i)\ni\kappa} x_{jri} + n_r(\kappa) \le \mathrm{cap}_r + z^A_{r\kappa}$$

$$\forall r,\ \kappa \in U_t \cup [t+1, t+H] \tag{6}$$

x_{jri} 는 작업 j 를 리전 r 에서 i 슬롯 뒤에 시작하게 두는 변수이고, $(j, i) \ni \kappa$ 는 그때 시점 κ 에 실행 중임을 뜻한다. 모든 대기 작업의 계획이 한 제약 안에서 함께 세어지므로, 여러 작업이 같은 리전의 같은 자리를 함께 차지하는 배정은 초과로 잡힌다.

예비 용량 ρ_r 은 다음 슬롯에 도착할 연기 불가 작업을 위해 남겨 두는 자리로, 미룰 수 있는 작업이 자리를 내주다 마감 직전으로 밀리는 일을 줄인다. 모든 리전 r 과 미래 슬롯 σ 에서 연기 가능 작업(Q^{def})의 계획은 상한보다 ρ_r 작은 $\mathrm{cap}_r - \rho_r$ 까지만 차지하게 하고, 이를 넘긴 양을

z^B 로 둔다. 예비 용량이 없어도 상한 위반은 생기지 않았다 (표 11).

$$\sum_{(j,i)\ni\sigma,\ j\in Q^{\text{def}}} x_{jri} + n_r(\sigma) \le \text{cap}_r - \rho_r + z^B_{r\sigma} \tag{7}$$

그림 3 은 한 리전에서 두 종류의 시점에서 센 동시 실행 수를 보인다. 회색은 이미 실행 중인 작업, 흰색은 이번 슬롯에 세운 계획, 빗금은 초과량이며, 점선은 연기 가능 작업의 계획에 적용되는 $\text{cap}_r - \rho_r$ 이다.

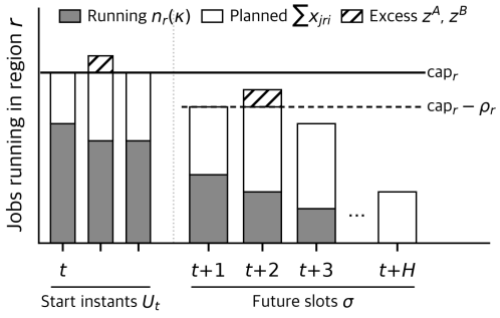


그림 3 한 리전의 용량 계산

Fig. 3 Capacity accounting in one region

3.6 통합 배정

통합 배정은 로드밸런서의 리전 후보와 스케줄러의 시작 후보를 묶어 (리전, 시작 시각) 쌍으로 고른다. 후보의 비용은 탄소 비용과 지연 비용을 가중치 α 로 더하고 실행 시간을 곱한 값이다.

$$c_j(r,i) = d_j\left[\alpha \tilde{C}_j(r,i) + (1-\alpha)\tilde{\ell}\left(o_j, r\right)\right] \tag{8}$$

α=1 이면 탄소만, α=0 이면 지연만 고려한다. 실행 시간을 곱하는 것은 긴 작업일수록 오래 배출하고 원격 리전을 오래 쓰기 때문이다. 슬롯 t 의 배정 문제는 식 (6), (7)을 제약으로 하는 다음 문제다.

$$\min \sum_{j\in Q}\sum_{r,i} c_j(r,i)x_{jri} + M_A \sum_{r,\kappa} z^A_{r\kappa} + M_B \sum_{r,\sigma\ge t+1} z^B_{r\sigma} + \varepsilon \sum_{j\in Q}\sum_{r,i} i\, x_{jri} \tag{9}$$

$$\sum_{r,i} x_{jri} = 1, \qquad \forall j \in Q \tag{10}$$

식 (10)은 한 작업을 한 자리에만 배정한다는 뜻이다. 정규화한 비용은 1 이하라 작업 하나의 배정 비용은 가장 긴 실행 시간 d_{max} 를 넘지 않으므로, M_B 는 d_{max} 의 2 배(하한 10), M_A 는 M_B 의 100 배로 두어 상한 초과를 예비 용량 침범보다 먼저 피하게 하였다. 마지막 항은 한 슬롯 미룰 때마다 $\varepsilon(10^{-4})$의 작은 대기 비용을 더해, 탄소와 지연의 비용 차이가 작으면 일찍 시작하는 쪽을 고르게 한다.

변수는 0~1 연속으로 두고 선형 계획(LP)으로 푼다. 용량 제약과 배정 제약이 함께 있어 해가 늘 정수로 나오지는 않으므로, 정수가 아닌 해가 나온 슬롯만 혼합 정수 계획(MILP)으로 다시 푼다. 풀고 나면 i=0 인 작업만 확정해 실행 현황에 반영하고, $i\ge$1 로 정해진 작업은 다음 슬롯에서 새 예측으로 다시 배정한다.

후보를 마감 안에서만 만들므로 마감 위반은 없고, 초과 변수가 있어 배정 문제는 항상 해를 가진다. 단, 이는 실행 시간을 제출 시점에 안다는 가정에 기댄다.

그림 4 는 한국에서 제출된 작업 하나에 대해 α 별로 고른 리전과 시작 시각이다. α=0 이면 한국에서 바로 시작하고, α=1 이면 예측이 가장 낮은 프랑스(12 gCO_2/kWh)에서 시작하지만 왕복 지연 시간이 238 ms 다. 무릎점 규칙은 이 슬롯에서 α=0.5 를 골랐다. 작업은 7 시간 뒤 일본에서 시작해 30 ms 의 지연으로 예측 탄소집약도를 352 에서 204 gCO_2/kWh 로 42% 낮췄다.

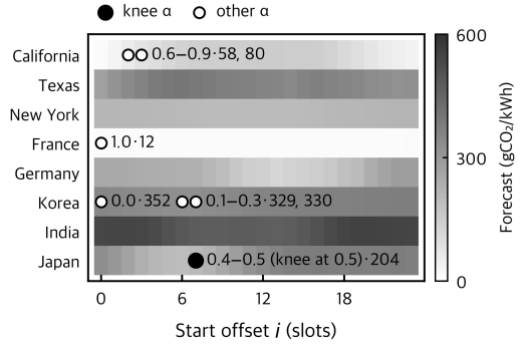


그림 4 α 에 따라 고른 리전과 시작 시각

Fig. 4 Region and start slot chosen at each α

3.7 가중치 규칙

α 는 슬롯마다 정한다. 후보 α 11 개($\mathcal{A}$ = {0, 0.1, …, 1})마다 배정 문제를 풀고, 그 배정의 실행 시간 가중 평균 왕복 지연 시간 $\bar{\lambda}_\alpha$ 와 예측 배출량 $\bar{e}_\alpha$ 를 구한다. 두 값을 후보 사이에서 0~1 로 정규화하면 이상점이 원점이 되며, 원점에 가장 가까운 후보를 고른다[23].

$$\alpha^* = \underset{\alpha\in\mathcal{A}}{\text{argmin}} \sqrt{\tilde{\lambda}_\alpha^2 + \tilde{e}_\alpha^2} \tag{11}$$

$\tilde{\lambda}_\alpha$ 와 $\tilde{e}_\alpha$ 는 정규화한 평균 지연과 예측 배출량이다. 이 규칙을 무릎점 규칙, 고른 점을 무릎점이라 부르며, 가중치를 미리 정하지 않아도 슬롯마다 탄소와 지연의 균형점을 고를 수 있다. 상한을 넘기는 후보가 있으면 초과가 가장 작은 후보들 안에서 고르고, 거리가 같으면

작은 α 를, 모든 후보의 지연이나 배출량이 같으면 α=1 을 고른다.

표 3 은 CAST 의 매개변수와 실험 값(기준 피크는 4.1.2 절), 알고리즘 1 은 한 슬롯의 처리 과정이다.

표 3 CAST 의 매개변수와 실험 값

Table 3 Parameters of CAST and their values in the experiments

Parameter	Symbol	Value
Slot length	—	1 h
Forecast horizon	H	24 h
Region ceiling	cap_r	1.25 × baseline peak, rounded up
Reserve	ρ_r	3 (synthetic), 0 (Azure)
Weight candidates	$\mathcal{A}$	0, 0.1, …, 1
Ceiling penalty	M_A	100 M_B
Reserve penalty	M_B	max(10, 2 × longest run in hours)
Early-start weight	ε	10^{-4}
MILP gap	—	0.01%

알고리즘 1 CAST의 슬롯별 배정
Algorithm 1 Per-slot assignment of CAST

Input: cap_r, ρ_r, H, M_A, M_B, ε, $\mathcal{A}$ = {0, 0.1, …, 1}, forecasts $\hat{C}_r^{(t)}(\cdot)$, job stream
Output: $(r(j), \tau_j)$ for all j

$Q \leftarrow \emptyset$
for $t \leftarrow 0, 1, 2, \ldots$ do
  for each j with $\lfloor s_j \rfloor = t$ do $Q \leftarrow Q \cup \{j\}$
  for each $j \in Q$ do
    $h_j \leftarrow \lfloor \min(D_j - d_j,\ t + H) - t \rfloor$
    if j is non-deferrable or $(D_j - d_j) - t < 1$ then $h_j \leftarrow 0$
  for each $\alpha \in \mathcal{A}$ do
    $x^\alpha \leftarrow$ LP solution of (6), (7), (9), (10)
    $(\bar{\ell}_\alpha, \bar{e}_\alpha) \leftarrow$ duration-weighted mean latency and forecast emissions of x^α
  $\alpha^* \leftarrow$ knee point by (11) among the candidates with minimum excess over the ceiling; $x \leftarrow x^{\alpha^*}$
  if x is fractional then $x \leftarrow$ MILP solution at α^* (HiGHS, relative gap 0.01%)
  for each (j, r) with $j \in Q$ and $x_{jr0} = 1$ do
    $r(j) \leftarrow r$; $\tau_j \leftarrow \max(t, s_j)$; add $[\tau_j, \tau_j + d_j)$ to the running set of r; $Q \leftarrow Q \setminus \{j\}$

4. 실험

4.1 실험 설정

4.1.1 탄소 자료와 예측기

8 개 리전의 2025 년 시간별 탄소집약도는 Electricity Maps[25]에서 받은 소비 기준 평균 배출계수다[26]. 리전별 구역은 US-CAL-CISO, US-TEX-ERCO, US-NY-NYIS, FR, DE, KR, JP, IN 이며, 일본과 인도는 국가 단위 구역이다. 예측은 과거 168 시간이 쌓이는 1 월 8 일부터 가능하므로, 1 년 전체를 시뮬레이션하되 1 월 8 일 이후에 제출된 작업만 평가한다(평가 기간 8,592 시간).

탄소집약도 예측기는 3.2 절의 LSTM 을 기본으로 쓴다. LSTM 은 2021~2023 년 자료로 학습하고 2024 년 자료로 검증했다. Adam(learning rate 0.001)과 batch size 64 로 학습했고, 검증 오차가 더 줄지 않으면 early stopping 을 적용했다(20~150 epoch). 비교를 위해 24 시간 전 같은 시각의 값을 쓰는 예측(SN-24h)과 실측값을 그대로 쓰는 완전예지(Oracle)도 쓴다. 2025 년의 평균 절대 오차는 LSTM 이 26.10 $\mathrm{gCO_2/kWh}$ 로, 24 시간 전 값(25.27)과 비슷하다. 이 차이는 하루 변동이 2024 년 11 월부터 두 배 가까이 커진 인도에서 오며, 인도를 빼면 LSTM 이 25.47 로 24 시간 전 값(27.24)보다 작다.

실험은 이번 슬롯에 도착할 작업, 이번 슬롯의 실측 탄소집약도, 캘리포니아·텍사스·독일 예측기에 넣는 앞으로 24 시간의 실측 기상값을 미리 안다고 가정한다. 이 가정을 뺀 결과는 4.4.3 절에 있다.

4.1.2 워크로드와 상한

실험에는 두 가지 워크로드를 쓰며, 표 4 에 비교했다(실행 시간은 1 년 전체 작업 기준). 합성 워크로드는 리전마다 하루 50 건의 작업을 생성한다. 작업은 허용 대기 시간에 따라 다섯 등급으로 나누고, 실행 시간은 등급마다 로그 표준편차 0.9, 0.9, 0.8, 0.7, 0.6 의 로그정규 분포에 상·하한을 두어 뽑는다(표 5). 도착 시각은 그림 5 와 같다. 등급 1~2 는 현지 2 시, 등급 3~5 는 현지 13 시를 중심으로 표준편차 3 시간의 정규분포에서 70%가 도착하고, 나머지 30%는 중심 시각에서 먼 12 시간에 고르게 도착한다. 현지 시각은 일광절약시간 없이 고정 시차로 둔다. 허용 대기 시간이 10 분 이하인 등급 3~5 의 작업은 대부분 연기 불가 작업이 된다.

Azure 워크로드는 Azure 의 2019 년 VM 기록[27, 28]에서 만들었다. 30 일 기록에 담긴 VM 약 270 만 개 가운데 기록의 처음과 끝에서 잘린 VM 을 빼고 146,016 개를 뽑아 1 년으로 이었다. 도착 시각과 실행 시간은 기록 그대로이고, 제출 리전은 8 개 리전에 고르게 나눈다. 등급은 실행 시간과 상관없이 합성과 같은 비율로 무작위로 주므로, 등급마다 실행 시간 분포가 같다(표 5). 두 워크로드는 작업 수가 거의 같지만, Azure 는 작업이 훨씬 길어 연기 불가 작업이 실행 시간의 절반을 차지하고 도착은 하루 내내 고르다.

표 4 두 워크로드

Table 4 The two workloads

Item	Synthetic	Azure
Jobs evaluated	143,200	143,165
Median run (s)	81	1,500
95th-pct run (s)	13,074	74,400
Non-deferrable run time (%)	1.3	51
Baseline peak	13	34
Ceiling	17	43
Reserve	3	0
Baseline emissions (kg)	28,625	213,513

표 5 등급별 허용 대기 시간과 실행 시간

Table 5 Allowed wait and run time by class

Class	Allowed wait	Synthetic run(s), median (min-max)	Azure run (s), median	Share (%)
1	12–24 h	5,400 (300–21,600)	1,500	30
2	2–8 h	1,800 (60–14,400)	1,500	10
3	2–10 min	180 (5–1,800)	1,500	10
4	15–60 s	30 (1–300)	1,500	20
5	1–5 s	5 (1–60)	1,500	30

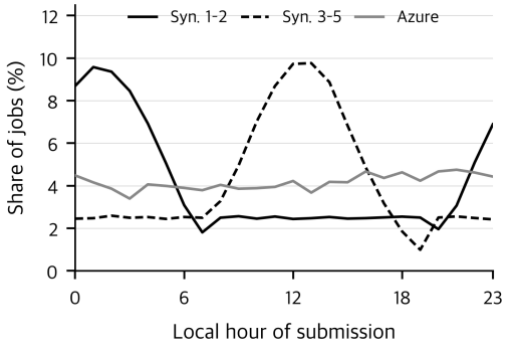


그림 5 현지 시각별 도착 비율

Fig. 5 Share of arrivals by local hour

사업자가 리전별 설비 규모를 공개하지 않으므로 상한은 실험 변수로 두고, 기본값은 기준 피크의 1.25 배를 올림한 값으로 모든 리전에 같게 주며(합성 17, Azure 43), 상한을 0.5~3 배로 바꾼 결과는 4.4.1 절에 있다. 기준 피크는 모든 작업을 제출 리전에서 바로 실행할 때 한 리전의 최대 동시 실행 수다. 예비 용량은 평가에서 제외한 첫 주(1 월 1~7 일)에 한 리전에서 동시에 실행된 연기 불가 작업의 최대 수로 정하며 합성에서는 3 이다. Azure 에서는 이 수의 대부분이 오래 실행 중인 VM 이라 새로 도착할 작업의 자리를 뜻하지 않으므로, 예비 용량을 0 으로 둔다. 이때 이용률, 곧 모든 작업의 실행 시간 합을 상한, 리전 수, 1 년의 곱으로 나눈 값은 합성 8.4%, Azure 26.1%다.

4.1.3 비교 대상

비교 대상은 2 장의 선행 연구 가운데 리전과 시각을 함께 옮기는 두 정책이다(표 6). Sukprasert 등[6]의 정책은 1 년치 실측 탄소집약도를 미리 안다고 보고, 공개된 탐색 코드를 1 시간 단위로 실행해 작업별 마감 안에서 탄소집약도가 가장 낮은 리전과 시각을 찾는다. Caspian[7]은 원문의 정식화를 선형 계획으로 풀어 구현하되, 1 시간 미만 작업은 실행 시간만큼만 슬롯 자원을 쓰게 하고 탄소는 LSTM 예측을 썼다. 용량은 슬롯마다 실행 시간의 합에 거는 하드 제약, 마감은 페널티로 두었고, 가중치는 원문대로 모두 1 이다. 마감을 1 시간 넘긴 작업은 제출 리전에서 바로 시작하게 했다. 모든 정책은 같은 자료와 배출 계산 방식으로 평가한다.

표 6 비교 대상

Table 6 Compared policies

Policy	Capacity	Deadline	Re-solve	Carbon
Sukprasert	None	Hard	Offline	Actual
Caspian	Hard	Soft	Hourly	LSTM
CAST	Soft	Hard	Hourly	LSTM

4.1.4 평가 지표

평가 지표는 표 7 과 같다. 두 실행을 같은 지연에서 비교할 때는 왕복 지연 시간 차이가 0.6 ms 안에 들도록 고정 α 를 맞추고, 95% 신뢰구간은 평가 기간을 7 일 단위로 묶어 다시 뽑는 블록 부트스트랩(2,000 회)으로 구한다.

표 7 평가 지표

Table 7 Metrics

Metric	Definition
Savings (%)	Emission reduction against running every job at once in its home region
Violations	Jobs started in a region already at its ceiling
Deadline miss	Jobs finished after their deadline
RTT (ms)	Mean round-trip latency, weighted by run time
Wait (h)	Mean time from submission to start
Req. cap.	Largest number of jobs running at once in one region

4.2 주요 결과

4.2.1 탄소 절감과 지연

CAST 는 두 워크로드 모두 상한 위반 없이 배출을 줄였고(표 8), 후보를 마감 안에서만 만들므로 마감 위반도 없었다(3.6 절). 무릎점 규칙은 α 를 슬롯마다 자동으로 골라, 모든 작업을 제출 리전에서 바로 실행할 때보다 합성 워크로드에서 71.93%, Azure 에서 51.35%를 줄였다.

상한이 없으면 작업이 저탄소 리전과 시각으로 몰린다. 같은 CAST 에서 용량 제약만 지우면 합성 무릎점에서 한 리전이 동시에 63 개, 곧 기준 피크 13 의 4.8 배를 실행하지만(α=1 에서는 9.0 배), CAST 는 상한 17 안에서 용량 제약이 없을 때보다 1.46%p 덜 줄일 뿐이다. 작업이 긴 Azure 에서는 저탄소 리전의 상한이 먼저 차서 이 차이가 11.35%p 로 커진다.

계절이 다른 네 주에서는 CAST 의 배출을 배출 하한과 비교했다. 배출 하한은 그 주 전체의 실측 탄소집약도와 작업을 미리 알고, 상한 17 과 CAST 와 같은 왕복 지연 시간 안에서 한 주를 한 번에 풀어 얻은 배출이다. CAST 무릎점은 이 하한보다 3.11~6.49%p 더 배출했다(α=1 은 0.33~1.47%p).

표 8 주요 결과

Table 8 Main results

Setting	Savings (%)	RTT (ms)	Wait (h)	Req. cap.	Viol.
Synthetic					
CAST	71.93	47.5	1.47	17	0
CAST (α=1)	95.65	121.3	2.42	17	0
No ceiling	73.39	49.3	1.44	63	24,993
No ceiling (α=1)	95.97	121.3	1.12	117	46,354
Azure					
CAST	51.35	39.3	2.25	43	0
CAST (α=1)	67.74	115.8	3.58	43	0
No ceiling	62.70	50.9	1.35	107	61,400
No ceiling (α=1)	95.11	120.1	0.89	203	139,527

4.2.2 가중치 α 와 무릎점 규칙

α 가 커질수록 작업이 저탄소 리전으로 옮겨 가고, 절감과 왕복 지연 시간이 함께 커진다(표 9). 표의 FR, CA, Moved 는 그 리전에서 실행된 몫과 옮긴 작업의 몫이며, 모두 실행 시간 기준이다. α 가 0.4 이하일 때는 옮긴 몫이 실행 시간의 25% 이하에 그치고, 0.5~0.7 에서 프랑스와 캘리포니아로 옮겨 가며 절감이 크게 오른다. α=1 에서는 실행 시간의 99.9%가 프랑스에서 실행되고 왕복 지연 시간이 121.3 ms 가 된다.

무릎점 규칙은 슬롯의 89%에서 0.5 또는 0.6 을 골랐고, 시간대별 평균 α 는 0.52~0.61 로 조금 움직인다. 대기 시간은 1.47 시간으로, α=0 을 뺀 어떤 고정 α 보다도 짧다.

무릎점 규칙을 왕복 지연 시간이 같은 고정 α(0.575)와 비교하면, 고정 α 가 1.86%p 더 줄이고 대기 시간은 1.63 시간으로 길다. 예측기, 상한, 작업을 바꾼 합성 9 개 조건 가운데 8 개에서 같은 방향이었고, Azure 에서는 차이가 신뢰구간 안이었다. 이 고정 α 는 1 년을 실행한 뒤에야 알 수 있는 값이며, 평가 전 첫 주 자료로 정한 고정 α(0.569)는 0.93%p 더 줄였다. 무릎점 규칙은 고정 α 보다 0.9~1.9%p 덜 줄이지만, α 를 고르기 위한 사전 실행 없이 첫 슬롯부터 쓸 수 있고 대기 시간이 짧다.

표 9 α 에 따른 배정(합성)

Table 9 Assignment by α (synthetic)

α	Savings (%)	RTT (ms)	Wait (h)	FR (%)	CA (%)	Moved (%)
0.0	0.00	0.0	0.00	12.3	12.7	0.0
0.1	23.48	0.5	1.89	16.8	12.7	4.5
0.2	26.61	1.2	1.90	22.5	12.7	10.2
0.3	29.25	2.3	1.95	23.9	15.0	14.0
0.4	38.64	9.1	1.92	27.7	20.0	24.3
0.5	62.99	32.0	1.73	43.2	24.7	47.5
0.6	77.69	53.7	1.58	53.4	32.9	64.7
0.7	88.99	78.7	1.52	65.2	33.6	75.4
0.8	93.51	98.2	1.74	81.1	18.9	80.6
0.9	95.24	113.7	2.11	93.9	6.1	85.1
1.0	95.65	121.3	2.42	99.9	0.1	87.7
Knee	71.93	47.5	1.47	49.2	31.2	59.0

그림 6 은 합성 워크로드에서 무릎점 규칙으로 작업이 옮겨 간 흐름을 보이며, 선 굵기와 아래 비율은 작업 건수 기준이다. 뉴욕, 독일, 인도의 작업은 주로 프랑스로, 텍사스, 일본, 한국의 작업은 주로 캘리포니아로 간다.

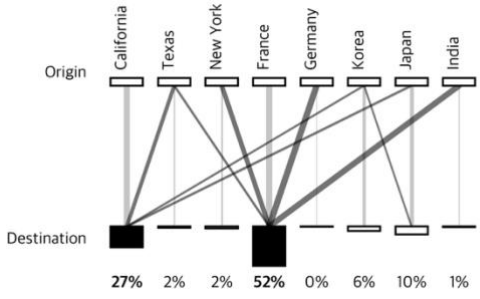


그림 6 리전 간 작업 이동

Fig. 6 Job flows between regions

4.3 선행 정책과의 비교

표 10 과 그림 7 은 합성 워크로드에서 두 선행 정책과 CAST 를 비교한 결과다. 두 정책은 상한 17 보다 큰 용량을 요구하거나 마감을 넘기고, CAST 는 둘 다 지킨다.

Sukprasert 등의 정책은 1 년치 실측 탄소집약도를 미리 알고 절감의 최대치를 구하는 분석으로, 96.63%를 줄이지만 작업 건수의 99.75%가 프랑스로 몰려 한 리전에 129 개, 곧 기준 피크의 9.9 배 용량이 필요하다. CAST(α=1)는 24 시간 예측으로 상한 17 안에서 95.65%(표 8)를 줄여, 그보다 0.98%p 덜 줄였다.

Caspian 은 67.54%를 줄이지만, 마감을 페널티로 두어 5,582 건(3.90%)이 마감을 넘긴다. 또 한 슬롯에 넣은 작업들의 실행 시간 합만 용량 안에 두므로, 본 논문처럼 동시 실행 수로 세면 짧은 작업이 같은 순간에 몰려 상한 17 을 넘는다(1,538 건, 필요 용량 34).

그림 7 의 (a)는 필요 용량, (b)는 왕복 지연 시간에 따른 절감이며, CAST 의 곡선은 (a)에서 상한을, (b)에서 α 를 바꾼 결과다. CAST 무릎점은 Caspian 보다 왕복 지연 시간이 짧으면서 더 많이 줄인다. Sukprasert 등의 정책은 같은 지연의 CAST(α=1)와 비슷하게 줄인다.

확정 시점만 도착 때로 바꾼 도착 시 확정 방식[8]과 두 방식의 왕복 지연 시간이 같아지는 고정 α 로 비교하면, CAST 가 기준 배출 대비 합성에서 0.96%p(95% 신뢰구간 0.51~1.46), Azure 에서 2.53%p(1.68~3.42) 덜 배출하며 둘 다 상한 위반은 없다. 이 차이는 상한이 빠듯할수록 커서 합성에서 상한 13 이면 1.46%p 이고, 넉넉한 26 에서는 신뢰구간이 0 을 포함한다. 프랑스로 옮긴 작업을 독일의 탄소집약도로 세는 한계 근사에서는 합성의 순서가 뒤집혀 도착 시 확정 방식이 0.89%p 덜 배출한다.

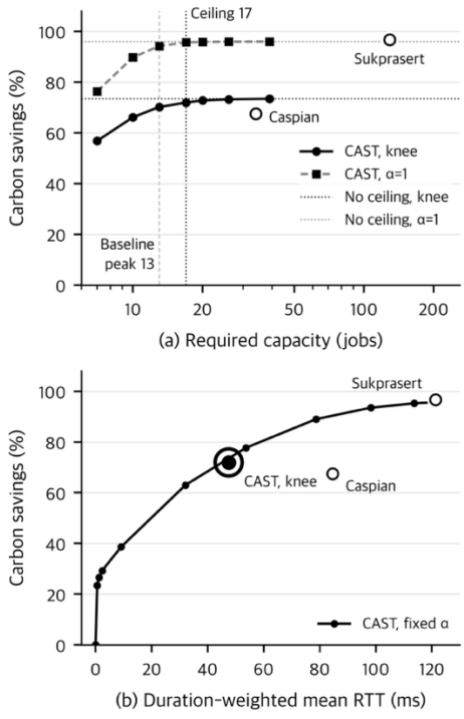


그림 7 선행 정책과의 비교: (a) 필요 용량, (b) 왕복 지연 시간

Fig. 7 Comparison with prior policies: (a) required capacity, (b) round-trip latency

표 10 선행 정책과의 비교(합성)

Table 10 Comparison with prior policies (synthetic)

Policy	Savings (%)	RTT (ms)	Req. cap.	Viol.	Deadline miss
Sukprasert	96.63	121.3	129	61,539	0
Caspian	67.54	84.5	34	1,538	5,582
CAST	71.93	47.5	17	0	0

4.4 조건별 결과

표 11 은 조건을 하나씩 바꾼 결과이며, 아래 소절과 같은 순서로 묶었다.

4.4.1 상한

합성에서는 상한을 바꾸어도 CAST 의 위반이 0 이다 총량을 113~114 로 두고 리전마다 자기 기준 피크에 비례하게, 또는 저탄소 세 리전(캘리포니아 · 프랑스 · 뉴욕)을 크게, 작게 나눈 세 배치에서도 위반은 없었다. 절감은 기준 피크의 2 배(26) 근처부터 거의 늘지 않는다 (표 11). Azure 에서는 상한을 기준 피크의 0.5 배(17)로 줄였을 때 465 건(0.32%)이 상한을 넘겼고, 465 건 모두 8 개 리전이 전부 상한까지 찬 순간에 시작했다.

표 11 조건별 결과

Table 11 Results under changed conditions

Condition	Knee (%)	α=1 (%)	Viol.
Base setting	71.93	95.65	0
Base setting, reserve 0	72.06	95.62	0
Ceiling			
Ceiling 7 (0.5×)	56.79	76.30	0
Ceiling 13 (1×)	70.16	94.18	0
Ceiling 26 (2×)	73.21	95.90	0
Ceiling 39 (3×)	73.42	95.96	0
Per-region ceiling (3 splits)	63.75–73.05	87.38–95.77	0
Azure, Ceiling 0.5×	19.70	—	465
Azure, Ceiling 0.6×	27.17	—	0
Azure, Ceiling 0.75–2×	34.15–61.89	—	0
Predictor			
SN-24h	64.44	95.59	0
Oracle	67.20	95.87	0
Workload			
5 workload draws	71.51 ± 0.43	95.40 ± 0.28	0
Deferrable 20–60%	70.01–72.54	93.35–95.70	0
Class 1–2 peak at 13 h	70.26	95.58	0
Arrivals ×2	69.45	92.58	0
Horizon 6–12 h	71.04–71.63	95.58–95.65	0
No future info, reserve 0	68.88	—	0
RTT cap			
50 ms	28.41	38.09	0
100 ms	32.98	46.83	0
150 ms	66.37	89.85	0

그림 8 은 캘리포니아의 7 월 7 일에 상한이 작동하는 모습으로, 가운데 칸의 가로선은 작업의 제출부터 마감까지, 막대는 CAST 의 실행 구간, 점선 상자는 상한이 없을 때의 실행 구간이다.

탄소집약도가 낮은 현지 여름 시각 9~16 시(UTC 16~23 시)에 CAST 의 동시 실행이 상한 17 에 이르고, 이 시간에 들어가지 못한 작업은 그 앞뒤 시각에 시작한다.

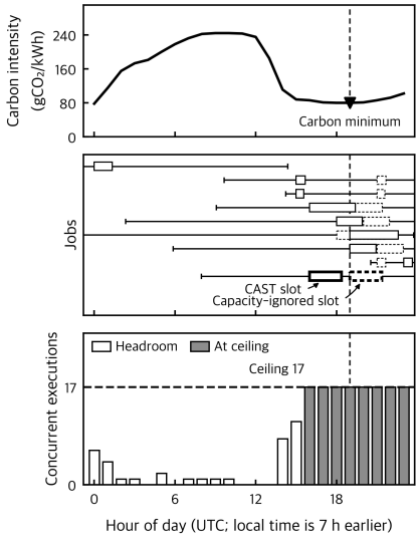


그림 8 캘리포니아의 하루(7 월 7 일)

Fig. 8 One day in California (7 July)

4.4.2 탄소집약도 예측기

탄소집약도 예측기를 바꾸어도 위반은 0 이고, α=1 의 절감은 세 예측기 사이에서 0.3%p 안에 있다. 무릎점 규칙에서는 예측기마다 고르는 α 가 달라 절감과 지연이 함께 움직이며, 완전예지의 무릎점 절감(67.20%)이 LSTM(71.93%)보다 낮은 것도 이 때문이다. 완전예지는 저탄소 구간이 뾰족해 정규화가 달라지고 무릎점이 더 낮은 α 를 고르며, 두 실행의 α 를 서로 바꿔 적용하면 절감 차 4.73%p 의 약 72%가 이 α 차이에서 온다.

4.4.3 작업과 미리 아는 정보

작업, 도착, 예측 구간을 바꾸어도 위반은 0 이고 무릎점 절감은 69.45~72.54%다. 바꾼 조건은 작업 재생성(5 회), 연기 가능 작업의 비율(20~60%), 등급 1~2 의 도착 중심(현지 2 시에서 13 시), 하루 도착 수(2 배), 예측 구간(6~12 시간)이다.

4.1.1 절의 세 가지, 곧 이번 슬롯에 도착할 작업, 이번 슬롯의 실측 탄소집약도, 세 리전의 실측 기상값을 미리 알지 않아도 위반은 0 이다. 이때 무릎점 절감은 예비 용량이 없는 기본 설정(72.06%)보다 3.18%p 준 68.88%다.

4.4.4 왕복 지연 상한

주 실험은 왕복 지연 상한을 두지 않아 리전 이동에 유리하다. 모든 작업에 50 ms 나 100 ms 의 상한을 두면 무릎점 절감이 28.41%, 32.98%로 준다. 100 ms 상한에서는 한국, 일본, 인도의 작업이 캘리포니아와 프랑스로 갈 수 없다.

4.4.5 계산 비용

CAST 의 풀이 시간은 슬롯 길이 1 시간보다 훨씬 짧다. SciPy 1.18.0 의 HiGHS 로 Apple M1 Max(메모리 32 GB)에서 실행했을 때, 후보 α 11 개를 모두 푼 슬롯당 시간은 합성에서 평균 223 ms, 최대 1,223 ms 다. 한 슬롯 문제의 변수는 가장 많을 때 7,848 개이고, 혼합 정수 계획으로 다시 푼 슬롯은 3.9%였다. Azure 는 슬롯당 평균 318 ms 이고, 합성의 도착을 두 배로 늘리면 변수가 15,816 개, 슬롯당 평균 556 ms 다.

4.5 배출 계산 방식

배정은 그대로 두고 배출만 다른 방식으로 다시 세면 표 12 와 같다. 실행 시간을 슬롯과 정확히 겹쳐 세거나, 옮긴 작업의 전송 에너지(1 GB 당 0.06 kWh[29])를 더하거나, 무탄소 전원에 전과정 배출 가정값(원자력 12, 재생 30 gCO_2 /kWh)을 넣거나, 프랑스와 캘리포니아의 설비 효율(PUE)을 바꾸어도 무릎점 절감은 66~75%에 머문다. 전송 에너지를 출발 리전으로 셀 때는 옮긴 작업마다 약 9 GB 를 보내야 리전 이동의 이득이 사라진다. 작업 전력을 Azure VM 의 코어 수에 비례하게 주어도 절감은 1.1%p 안에서 움직이고 정책 간 순위는 바뀌지 않는다.

다만 옮겨 온 작업 때문에 추가로 돌리는 발전원의 배출, 곧 한계 배출로 세면 절감이 크게 준다. 프랑스에 늘어난 수요를 이웃한 독일의 발전이 댄다고 보고 프랑스로 옮긴 작업을 같은 시각 독일의 탄소집약도로 세면 무릎점 절감은 38.99%(Azure 22.37%)이고, α=1 은 작업을 옮기지 않고 시각만 옮긴 경우(18.78%)보다도 낮다. 선행 정책과의 비교(표 10)는 평균 배출계수로만 셌다.

표 12 배출 계산 방식별 CAST 의 절감(합성, %)

Table 12 CAST savings by emission accounting (synthetic, %)

Accounting	Knee	α=1
Average factor	71.93	95.65
Exact overlap	71.80	95.64
Transfer 1 GB, destination	71.16	95.20
Transfer 1 GB, origin	66.01	87.65
Life-cycle approx.	67.07	90.28
Marginal, France only	38.99	18.11
PUE 1.6 / 1.1 at FR, CA (reserve 0)	68.66 / 74.54	—

5. 결론 및 향후 연구

탄소집약도를 따라 작업의 리전과 시각을 옮기면 작업이 저탄소 리전과 시각으로 몰려 리전 용량을 넘기기 쉽다. 선행 연구는 용량과 마감 가운데 하나를 제약으로 두지 않거나, 작업이 도착할 때 배정을 확정하고 다시 풀지 않는다. 본 논문은 매시간 대기 중인 모든 작업의 리전과 시작 시각을 하나의 선형 계획 문제로 다시 풀고 이번 슬롯에 시작하는 작업만 확정하는 CAST 를 제안하였다. CAST 는 마감을 하드 제약으로 두고, 용량은 상한을 넘긴 양에 가장 큰 페널티를 주는 소프트 제약으로 두며, 탄소와 지연 사이의 가중치는 미리 정하지 않고 슬롯마다 무릎점 규칙으로 고른다.

8 개 리전의 2025 년 1 년치 탄소집약도 위에서 평균 배출계수로 배출을 계산하면, CAST 는 합성 워크로드의 배출을 71.93%(α=1 에서 95.65%), Azure VM 기록으로 만든 워크로드의 배출을 51.35%(α=1 에서 67.74%) 줄였고, 두 워크로드 모두 상한 위반과 마감 위반이 없었다. 같은 CAST 에서 용량 제약만 지우면 합성에서 한 리전에 기준 피크의 4.8~9.0 배가 몰렸다.

같은 자료에서 선행 정책과 비교하면, Sukprasert 등의 정책은 1 년치 실측 탄소집약도를 미리 알고도 한 리전에 기준 피크의 9.9 배 용량을 요구했고, CAST(α=1)는 24 시간 예측으로 상한 안에서 그보다 0.98%p 덜 줄였다. Caspian 은 CAST 보다 적게 줄이면서 5,582 건의 마감을 어겼다. 확정 시점만 다른 도착 시 확정 방식과 같은 왕복

지연 시간의 고정 α 로 비교하면, CAST 가 합성에서 0.96%p, Azure 에서 2.53%p 덜 배출하였다. 이는 탄소 인지 스케줄링의 절감을 그 배정이 요구하는 리전 용량, 마감 준수 여부와 함께 평가해야 함을 보여준다.

이 결과는 다음 조건에서 얻은 것이다. 실험은 이번 슬롯에 도착할 작업, 이번 슬롯의 실측 탄소집약도, 세 리전의 실측 기상값을 미리 안다고 두었으며, 이 셋을 빼면 위반은 그대로 0 이고 무릎점 절감은 68.88%였다. 주 실험은 왕복 지연 상한을 두지 않아 리전 이동에 유리하며, 모든 작업에 100 ms 상한을 두면 무릎점 절감은 32.98%다. 배출은 평균 배출계수로 셌고, 프랑스로 옮긴 작업을 독일의 탄소집약도로 세는 한계 근사에서는 무릎점 절감이 38.99%로 줄며 도착 시 확정 방식과의 순서도 합성에서 뒤집힌다. 평가는 2025 년 한 해 자료의 시뮬레이션이며, Azure VM 을 미룰 수 있는 배치 작업으로 보고 실행 시간을 제출 때 안다고 두었고, 작업 전력은 모두 1 kW, 용량은 동시에 실행되는 작업 수로 셌다. 선행 정책은 같은 자료 위에서 다시 구현했으므로, 원 논문의 수치를 재현해 확인하지는 못했다.

향후에는 CAST 를 실제 클러스터 관리 계층에 연동해 배정을 바꾸고 작업을 옮기는 데 드는 비용을 확인하고, 한계 배출계수 자료로 다시 평가할 계획이다. 또한 작업마다 다른 왕복 지연 상한과, CPU · GPU 처럼 자원과 전력이 다른 작업을 용량 제약에 함께 반영하는 방향으로 확장할 수 있다.

참 고 문 헌

[1] International Energy Agency, *Energy and AI, World Energy Outlook Special Report*. IEA, Paris, Apr. 2025.

[2] European Parliament and Council, "Directive (EU) 2023/1791 of the European Parliament and of the Council of 13 September 2023 on Energy Efficiency and Amending Regulation (EU) 2023/955 (recast)," *Official Journal of the European Union*, L 231, pp. 1-111, Sep. 2023.

[3] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, "Carbon-Aware Computing for Datacenters," *IEEE Transactions on Power Systems*, Vol. 38, No. 2, pp. 1270-1280, Mar. 2023.

[4] A. Souza, S. Jasoria, B. Chakrabarty, A. Bridgwater, A. Lundberg, F. Skogh, A. Ali-Eldin, D. Irwin, and P. Shenoy, "CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services," *Proc. of the 14th Int. Green and Sustainable Computing Conf. (IGSC)*, pp. 67-73, Oct. 2023.

[5] D. Maji, B. Pfaff, V. P R, R. Sreenivasan, V. Firoiu, S. Iyer, C. Josephson, Z. Pan, and R. K. Sitaraman, "Bringing Carbon Awareness to Multi-cloud Application Delivery," *Proc. of the 2nd Workshop on Sustainable Computer Systems (HotCarbon)*, pp. 1-6, Jul. 2023.

[6] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, "On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud," *Proc. of the 19th European Conf. on Computer Systems (EuroSys)*, pp. 924-941, Apr. 2024.

[7] T. Bahreini, A. N. Tantawi, and O. Tardieu, "Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments," *Proc. of the 32nd Int. Conf. on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pp. 1-8, Oct. 2024.

[8] M. Zanotto, L. Vicentini, R. Vreto, F. Lumpp, D. Braga, and S. Fiore, "Toward a User-centric Kubernetes-based Architecture for Green Cloud Computing," *Future Generation Computer Systems*, Vol. 186, Article 108749, Jan. 2027 (in press).

[9] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud," *Proc. of the 22nd Int. Middleware Conf. (Middleware)*, pp. 260-272, Dec. 2021.

[10] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, "CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency," *Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, Vol. 7, No. 3, Article 57, Dec. 2023.

[11] W. A. Hanafy, Q. Liang, N. Bashir, A. Souza, D. Irwin, and P. Shenoy, "Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions," *Proc. of the 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 3, pp. 479-496, Apr. 2024.

[12] Z. Liu, M. Lin, A. Wierman, S. H. Low, and L. L. H. Andrew, "Greening Geographical Load Balancing," *Proc. of the ACM SIGMETRICS Joint Int. Conf. on Measurement and Modeling of Computer Systems (SIGMETRICS)*, pp. 233-244, Jun. 2011.

[13] M. Lin, Z. Liu, A. Wierman, and L. L. H. Andrew, "Online Algorithms for Geographical Load Balancing," *Proc. of the 2012 Int. Green Computing Conf. (IGCC)*, pp. 1-10, Jun. 2012.

[14] S. Ren, Y. He, and F. Xu, "Provably-Efficient Job Scheduling for Energy and Fairness in Geographically Distributed Data Centers," *Proc. of the 32nd IEEE Int. Conf. on Distributed Computing Systems (ICDCS)*, pp. 22-31, Jun. 2012.

[15] Í. Goiri, K. Le, M. E. Haque, R. Beauchea, T. D. Nguyen, J. Guitart, J. Torres, and R. Bianchini, "GreenSlot: Scheduling Energy Consumption in Green Datacenters," *Proc. of the Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC)*, pp. 1-11, Nov. 2011.

[16] G. Attenni, Y. Moawad, N. Bartolini, and L. Thamsen, "Spatio-Temporal Shifting to Reduce Carbon, Water, and Land-Use Footprints of Cloud Workloads," arXiv:2512.08725, Dec. 2025.

[17] A. Lechowicz, N. Christianson, B. Sun, N. Bashir, M. Hajiesmaili, A. Wierman, and P. Shenoy, "Learning-Augmented Competitive Algorithms for Spatiotemporal Online Allocation with Deadline Constraints," *Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, Vol. 9, No. 1, Article 8, pp. 1-49, Mar. 2025.

[18] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, "Flex: Grid-Responsive Provisioning and Scheduling for Elastic Cloud

Clusters," *Proc. of the 2026 ACM Symp. on Cloud Computing (SoCC)*, Nov. 2026 (to appear; arXiv:2505.18357).

[19] S. P. Sethi and G. Sorger, "A Theory of Rolling Horizon Decision Making," *Annals of Operations Research*, Vol. 29, No. 1, pp. 387-415, Dec. 1991.

[20] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained Model Predictive Control: Stability and Optimality," *Automatica*, Vol. 36, No. 6, pp. 789-814, Jun. 2000.

[21] S. Chand, V. N. Hsu, and S. Sethi, "Forecast, Solution, and Rolling Horizons in Operations Management Problems: A Classified Bibliography," *Manufacturing & Service Operations Management*, Vol. 4, No. 1, pp. 25-43, Jan. 2002.

[22] L. Chen, F. Eberle, N. Megow, K. Schewior, and C. Stein, "A General Framework for Handling Commitment in Online Throughput Maximization," *Mathematical Programming*, Vol. 183, No. 1-2, pp. 215-247, Sep. 2020.

[23] M. Zeleny, "Compromise Programming," in J. L. Cochrane and M. Zeleny (Eds.), *Multiple Criteria Decision Making*, pp. 262-301, University of South Carolina Press, Columbia, 1973.

[24] Microsoft. Azure Network Round-Trip Latency Statistics [Online]. Available: https://learn.microsoft.com/en-us/azure/networking/azure-network-latency (downloaded 2026, Jul. 20)

[25] Electricity Maps. API Documentation [Online]. Available: https://app.electricitymaps.com/docs (downloaded 2026, Oct. 18)

[26] Electricity Maps. Methodology [Online]. Available: https://www.electricitymaps.com/data/methodology (downloaded 2026, Jul. 20)

[27] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms," *Proc. of the 26th ACM Symp. on Operating Systems Principles (SOSP)*, pp. 153-167, Oct. 2017.

[28] Microsoft Azure. Azure Public Dataset V2 (2019 VM Trace) [Online]. Available: https://github.com/Azure/AzurePublicDataset (downloaded 2026, Jul. 20)

[29] J. Aslan, K. Mayers, J. G. Koomey, and C. France, "Electricity Intensity of Internet Data Transmission: Untangling the Estimates," *Journal of Industrial Ecology*, Vol. 22, No. 4, pp. 785-798, Aug. 2018.